QR Image Transfer

Image Transmission via a Sequential QR Stream over a Screen–Camera Link: from Fixed-Index Loops to LT Fountain Codes

A build-and-measure engineering study with field data from commodity phones

October 2026 · Web-to-Web implementation (laptop display → phone camera)

Abstract. We design, implement and measure a system that moves arbitrary binary data — demonstrated on images up to 3.6 MB — through a one-way optical channel: a laptop browser displays a time-multiplexed stream of QR codes while a phone browser scans, reassembles and verifies the file. Starting from a naive fixed-index loop (224 s for 33.5 kB, stuck at 95%), three diagnosis-driven fixes — integer-scale QR rendering, LT fountain coding, and metadata interleaving — cut the same transfer to 18 s and then 10 s (22×), and delivered a 3,594,667-byte photograph in 1,146 s with SHA-256 match. Every claim is backed by timestamped receiver logs (7 sessions, >30,000 decode attempts). We document the QR symbology, the screen–camera channel, fountain-code theory with equations, the full protocol, the experimental method, and the physical limits we hit (V25-Q undecodable on the test sensor).

1 Introduction

Moving data between a screen and a camera without any radio, cable, pairing or network is an old idea with renewed relevance for anyone who values a channel that is visible, directional and infrastructure-free. Think air-gapped machines, exhibition demos, or onboarding without credentials. The simplest embodiment — show a sequence of QR codes as a continuous stream, film it with a phone — is easy to sketch and surprisingly hard to make fast and reliable.

This paper reports a complete build-and-measure cycle. **Contribution 1** is a working Web-to-Web system (no installs, no backend) with two protocols: v1 fixed-index chunks and v2 LT fountain drops. **Contribution 2** is a measurement study: 7 instrumented sessions, timestamped per-frame arrivals and decoder error histograms, showing exactly where time goes. **Contribution 3** is a reproducible diagnosis method: each bottleneck was identified from log statistics (including a 10^{-14} impossibility proof that two frames were systematically unreadable) and fixed with a targeted change, yielding a 22× speedup on identical hardware.

2 What a QR Code Actually Is

The QR code is a matrix symbology standardised in ISO/IEC 18004 [1], invented at DENSO WAVE [2]. A symbol is a square of black/white *modules* whose side length is fixed by the *version* V = 1..40:

$$M = 4V + 17 \quad \text{modules per side}$$

so V1 is 21×21 and V40 is 177×177. Three 7×7 *finder patterns* (position detection), separators, *timing patterns* (alternating row/column that sets the module grid), format information (ECC level + mask, itself BCH-protected), an alignment pattern grid (V≥2), and the remaining modules carry interleaved data and error-correction codewords (Fig. 1). The physical module pitch is

$$m = W/(4V + 17),$$

with W the displayed symbol width — the single most important equation in this paper, as Section 9.1 shows.

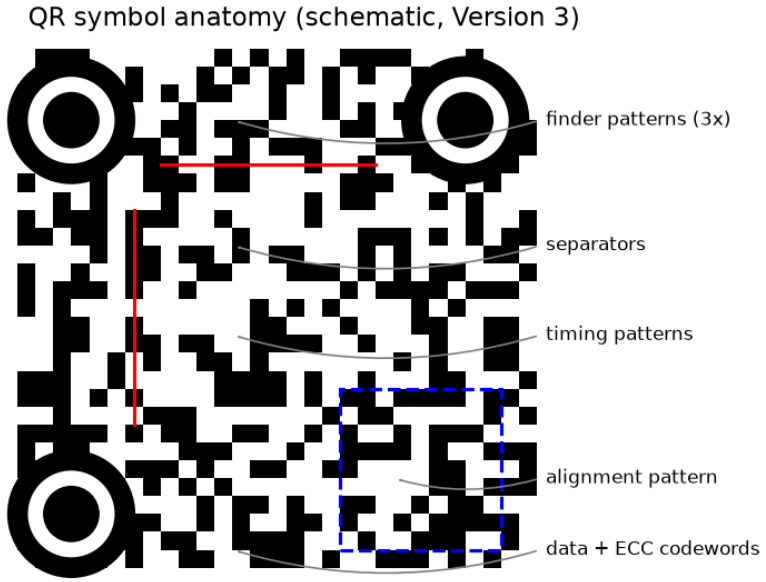


Figure 1 — QR symbol anatomy (schematic). Only the white area carries payload; everything else is overhead that makes the symbol findable and self-clocking.

2.1 Capacity

Capacity depends on version, encoding *mode* (numeric 3■ bits/digit, alphanumeric 5.5 bits/char, byte 8 bits) and error-correction level. The hard ceilings at V40-L are 7,089 numeric / 4,296 alphanumeric / **2,953 bytes** / 1,817 Kanji [1, 18]. Byte-mode capacities (the mode used here) grow roughly quadratically with V (Fig. 2):

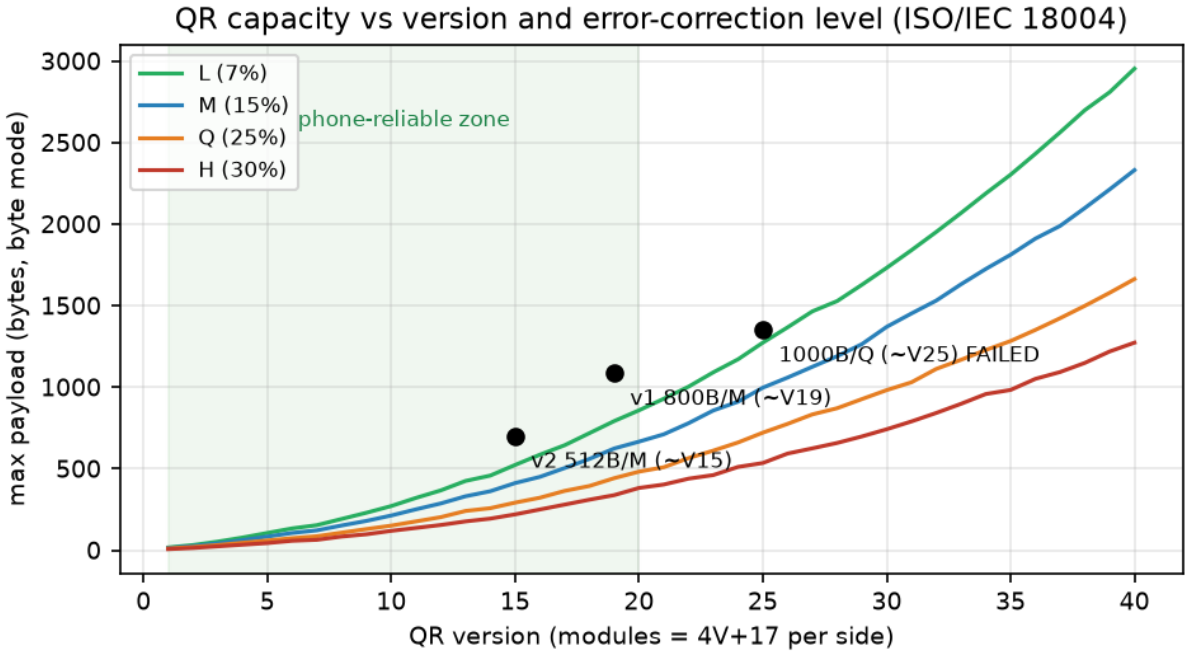


Figure 2 — Byte-mode capacity vs version and ECC level [1, 18]. Green band: reliably phone-decodable. Dots: this project's operating points; V25-Q (failed run, Sec. 9.4) sits outside the test sensor's optics.

V	L (7%)	M (15%)	Q (25%)	H (30%)
1	17	14	11	7
5	106	84	60	44
10	271	213	151	119
15	523	412	292	220
20	858	666	482	382
25	1,273	997	721	535
30	1,732	1,370	982	742
35	2,303	1,812	1,283	983
40	2,953	2,331	1,663	1,273

Table 1 — Byte-mode capacities (excerpt, ISO/IEC 18004 capacity table).

Two warnings matter more than the ceilings. First, *capacity you cannot scan is not capacity* [18]: a V40 symbol at 5 cm has 0.27 mm modules, below phone-camera resolution at normal distances. Practical streaming work uses mid versions (e.g. ≤213 bytes at V10-M). Second, raising ECC roughly halves capacity (V40: 2,953 → 1,273 bytes L→H) because redundancy is spent from the same codeword budget.

2.2 Reed–Solomon error correction

Each QR block protects k data codewords with a Reed–Solomon [20] code over GF(2^8), appending n–k parity codewords that correct up to t errors:

$$n - k = 2t,$$

with four selectable levels (Table 2). Interleaving spreads burst errors across blocks. Our receiver adds two more integrity layers on top: CRC-16 per frame and SHA-256 [FIPS 180-4] per file.

Level	Recovery	Use here
L	~7% codewords	—
M	~15% codewords	default (density vs robustness sweet spot)
Q	~25% codewords	tested; V25-Q still optically unresolvable (Sec. 9.4)
H	~30% codewords	— (capacity cost too high)

Table 2 — QR error-correction levels [1].

3 The Screen–Camera Channel

Streaming QR is an instance of screen–camera communication (SCC), surveyed recently with reported throughputs up to ~900 kbps for streaming codes (7.67 Mbps distortion-free) [8]. The channel is hostile in specific, well-documented ways: the display rate must stay well under half the camera capture rate or rolling-shutter and inter-frame mixing create undecodable composite frames [10, 11]; perspective, moiré, defocus and auto-exposure all attack the module grid; and phone CPUs decode only a few frames per second in JavaScript, so the sampler — not the display — sets the pace (our measured 14–18 attempts/s, Sec. 8).

System	Setup	Reported throughput
Xie et al. (phone↔phone, 2 MP front cam) [8]	bidirectional, low-res	~3 kbps
Vanderhulst & Trappeniers [8]	video loop, V1→V40 sweep	336 bps → 472 kbps
Liu et al. 2020, dynamic QR [9]	mobile, real- / non-real-time	150 / 300 kbps
Liu et al. 2020, colour-multiplexed [9]	3× QR per frame	320 / 900 kbps

System	Setup	Reported throughput
Fath et al., colour QR + temporal FEC [8]	screen at ½ camera rate	120.43 kbps
FareQR, border + Viterbi de-mixing [10]	rolling-shutter/mixing fix	block-error reduction
RescQR, mixture separation + Viterbi [11]	standard QR	400+ kbps goodput
ChromaCode / AirCode, imperceptible [12, 13]	invisible + audio metadata	~0.55 / ~1 Mbps
This work (v2 + integer modules)	browser-to-browser, mono QR	~3.1 kB/s net (≈25 kbps), 3.6 MB file

Table 3 — Prior screen–camera results and this work. Ours is slower per second but needs no installs, no colour calibration and no back-channel — and ships with per-frame telemetry.

Receiver technology constrains the design: @zxing/browser BrowserQRCodeReader [14] and the pure-JS port jsQR [15] (best run in a Web Worker [16]) both degrade on full-HD inputs, so downscaling to ~640–960 px before decode is standard practice [17]. Camera access further mandates a secure context (HTTPS), which shaped our LAN test rig (Sec. 7).

4 Data Preparation: Compression and Integrity

A deployed image (JPEG/PNG/WebP) is already entropy-coded; generic gzip on top saves ~0%. The protocol therefore transmits raw file bytes. The *effective* compression lever is semantic: optional sender-side downscale + WebP re-encode (typically 5–10× smaller, visually near-lossless for previews), kept as a user choice so original-quality transfer stays bit-exact. Text-mode framing adds a fixed 15-byte binary header plus base64url transport encoding:

$$L_{out} = 4\lceil L_{in}/3 \rceil \quad (+33\%\ \text{overhead}),$$

e.g. an 815-byte v1 frame becomes 1,088 characters (≈V19-M); a 526-byte v2 drop becomes ≈700 characters (≈V15-M). Integrity is layered: Reed–Solomon inside each symbol, CRC-16-CCITT per frame (rejects consistent 1-bit misreads before they can poison reassembly), and SHA-256 over the reassembled file before anything is shown or saved. No SUCCESS is declared on hash mismatch — the receiver simply keeps scanning.

5 Fountain Codes: Theory

Fixed-index streaming is a coupon-collector process: to collect N specific frames with a sampler delivering λ novel frames/s,

$$E[T] = N \cdot H_N/\lambda, \qquad H_N \approx \ln N + \gamma,$$

and the tail dominates — for N = 500 at λ = 3/s the expectation exceeds 1,000 s, all spent waiting on a handful of specific frames (Sec. 9.2 measures exactly this). LT codes [4], the practical form of the digital fountain idea [5] (see also [6, 7, 21]), remove the notion of a *specific* frame: the sender emits an endless stream of XOR combinations ('drops') of the K source blocks; the receiver recovers after *any*

$$M = K(1+\varepsilon)$$

drops (ε ≈ 0.05–0.3), via the peeling (belief-propagation) decoder: repeatedly resolve degree-1 drops, XOR them out of pending drops, cascade. Drop degrees follow Luby's robust soliton distribution,

$$\rho(1) = 1/K, \quad \rho(d) = 1/[d(d-1)],$$

$$\tau(d) = R/(dK)\ (d < S), \quad \tau(S) = R\ln(R/\delta)/K, \quad R = c\ln(K/\delta)\sqrt{K},$$

normalised to μ = (ρ+τ)/Z (Fig. 3 shows our K = 66 instance: the degree-1 spike that starts peeling, the 1/[d(d−1)] tail, and the τ bump guaranteeing coverage). Degrees and neighbours are derived deterministically from a 32-bit seed, so only (seed, payload) crosses the wire. Our sender transmits the K systematic (degree-1) drops first — bulk speed of v1 — then endless fresh pseudorandom drops, so the tail is mathematically impossible: every received drop helps regardless of which ones came before.

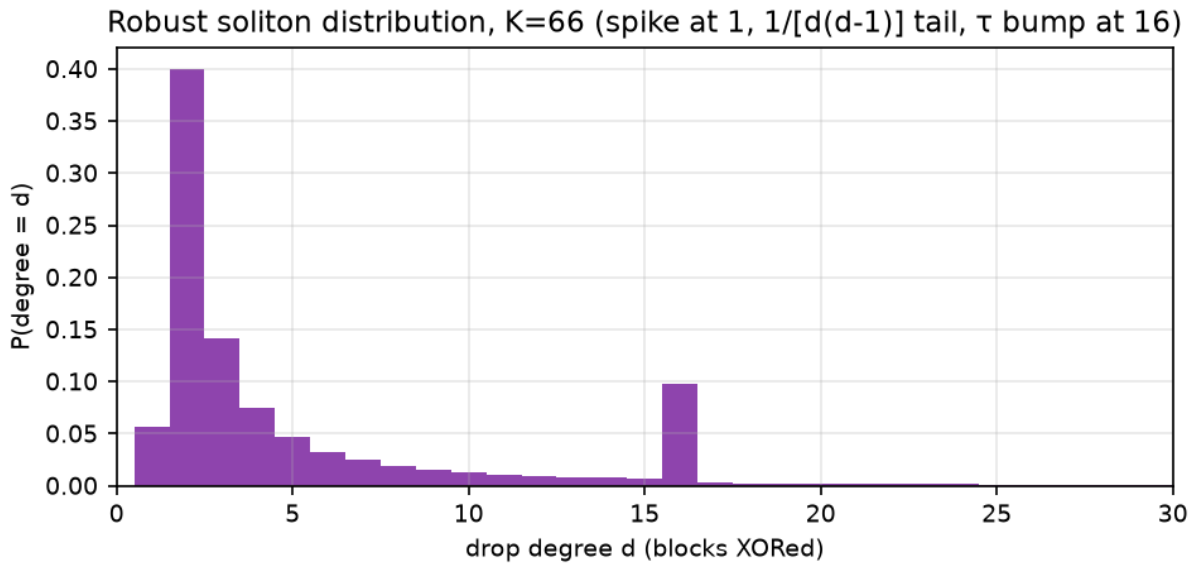


Figure 3 — Robust soliton degree distribution used (K = 66, c = 0.1, δ = 0.5).

6 System Design

Two static pages, no backend, served over LAN HTTPS. **Sender** (laptop): file/text → ArrayBuffer → split → QR encode (qrcode [19]) → fullscreen high-contrast loop with chained scheduling (each frame fully rendered before the next interval starts, guaranteeing full dwell). **Receiver** (phone): getUserMedia (environment camera, capped 960×720 for 3–5× faster decode) → continuous ZXing decode → dedupe map → progress/missing UI → concat → SHA-256 verify → preview/download. A one-tap .txt session report (per-drop timestamps, 2-s counter samples, error histogram) made every result in this paper analysable offline.

Field	v1 DATA (15 B + payload)	v2 drop (14 B + payload)
Magic / ver	'QT' / 0x01	'QF' / 0x01
Identity	fileId u32	fileId u32
Sizing	total u24	K u24 (blocks)
Position	index u24	seed u32 (sys if < K, else LT flag + counter)
Check	CRC-16 of payload	— (peeling + SHA-256; bad drops naturally stall)
Transport	base64url	base64url
META (JSON)	{v, fileId, name, size, mime, total, chunkSize, sha256}	same + {proto:2, K, blockSize}; repeated every 20 frames

Table 4 — Wire formats. Full header spec in the project methodology document.



Figure 4 — Frame layouts to scale by field width (payload truncated).

Key robustness properties: sessions key off (proto, fileId, total/K) from *any* frame, so mid-loop joins collect immediately; repeat META frames never wipe progress; a new fileId resets cleanly; post-completion loop repeats are ignored; and every v2 loop pass carries fresh LT seeds, so the stream never repeats a drop.

7 Experimental Method

Hardware. Sender: commodity laptop (max brightness, dark UI surround, fullscreen). Receiver: Android 10 phone, Chrome Mobile 155, handheld at ~20–40 cm, perpendicular, office indoor light. **Fixtures.** 33,509-byte PNG screenshot (K = 42–66 depending on block size) for iteration; 3,594,667-byte JPEG photo (K = 4,494 at 800 B) for the scaling run. **Factors.** chunk/block {400 … 1200 B}, display fps {4 … 12}, ECC {L, M, Q}, QR px {560, 720}, 2× dwell on/off, protocol v1/v2. **Metrics.** time-to-complete (TTC), novel rate, decode ok/attempts, fountain overhead (drops–K)/K, final missing set, SHA-256 verdict — all from receiver logs, n = 1 run per cell for engineering iteration (pre-registered pass/fail in the methodology doc).

8 Results

Run	Config	K/N	Drops	Overhead	TTC	Decode ok/att	Outcome
v1 baseline	800 B, 8 fps, M, 720, 2×	42	—	—	224 s	1,748/3,690 (47%)	DONE; 3 min stuck at 95%
v2 fountain	512 B, 8 fps, M, 720	66	85	29%	82 s	1,018/1,580	DONE; data 13 s + META wait 60 s
v2 + META×512 B	512 B, 8 fps, M, 720	66	82	24%	18 s	165/326	DONE, clean cascade
v2 speed	800 B, 10 fps, M, 720	42	47	12%	10 s	85/180 (47%)	DONE
v2 photo	800 B, 10 fps, M, 720	4,494	5,931	32%	1,146 s	11,080/20,826 (53%)	DONE 3.6 MB, SHA match
v2 dense-Q	1000 B, 10 fps, Q, 720	34	10	—	stuck 0/34	12/547 (2%)	FAILED: optical wall

Table 5 — All measured runs, same 33.5 kB image except the photo row (3.6 MB). Same phones, same room.

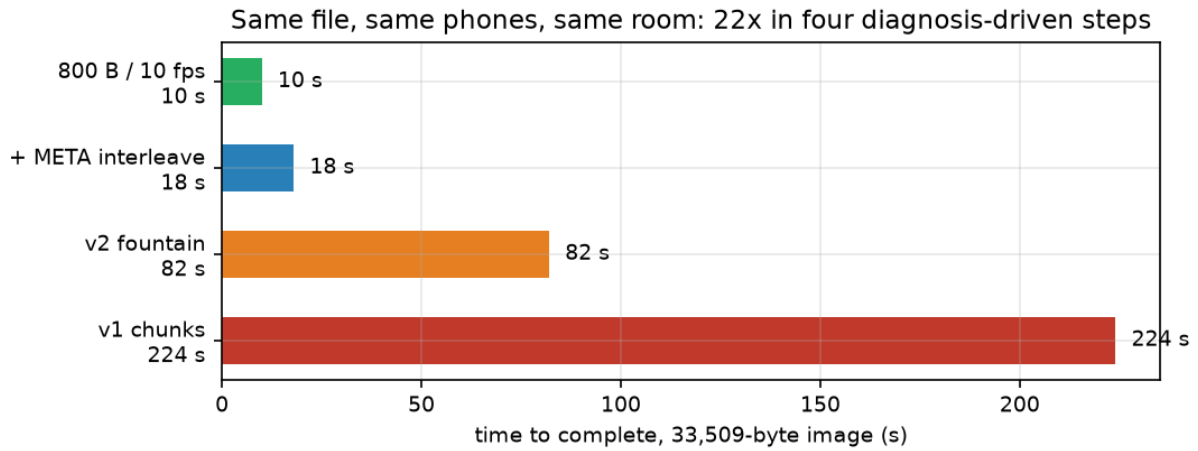


Figure 5 — 224 s → 10 s (22×) on the identical file: each bar is one diagnosis-driven fix.

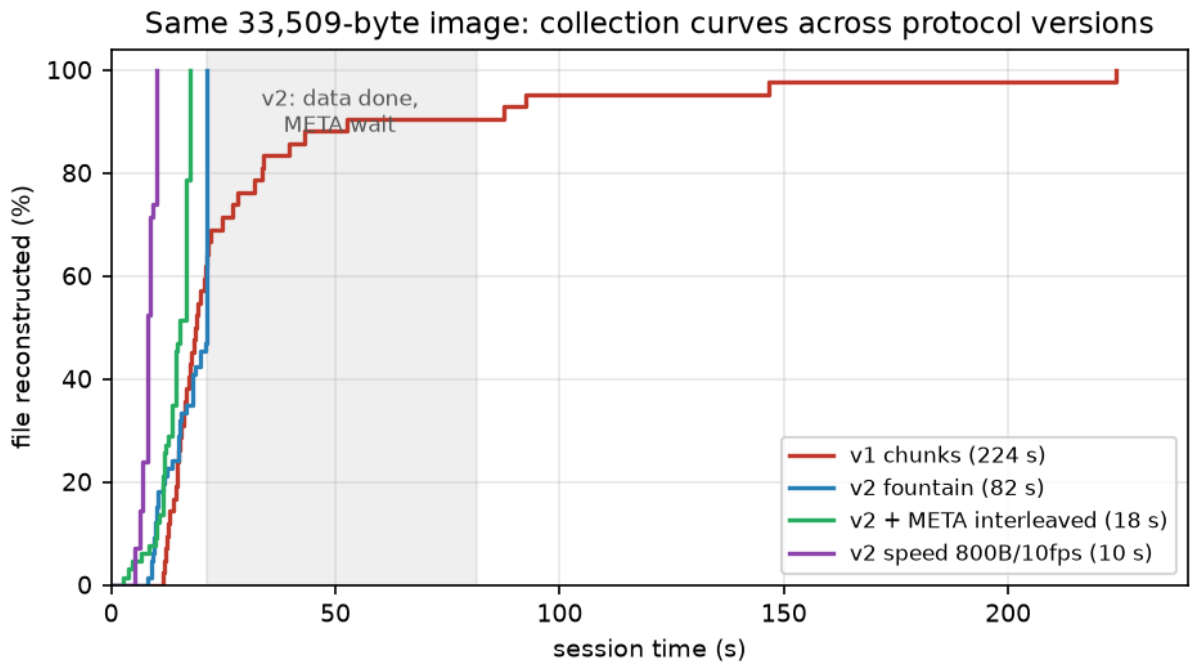


Figure 6 — Collection curves, same file. v1 crawls the tail; v2 finishes data in 13 s but waits 60 s on a once-per-pass META (shaded); interleaving META removes the wait.

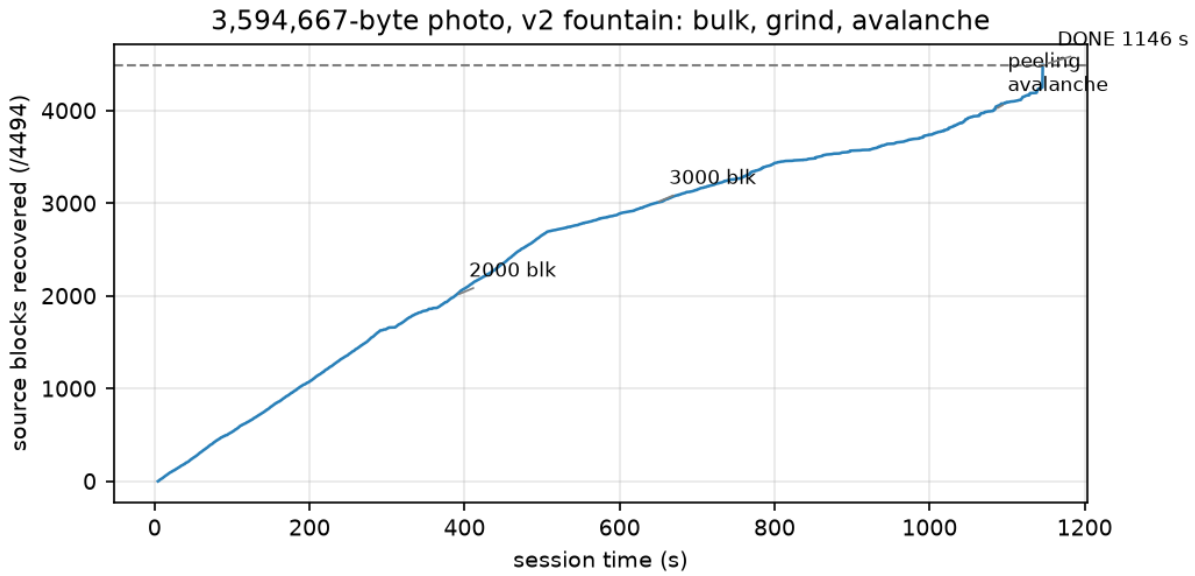


Figure 7 — 3.6 MB photo: linear bulk (~5/s), mid grind, then the peeling avalanche 4,003 → 4,494 in 64 s.

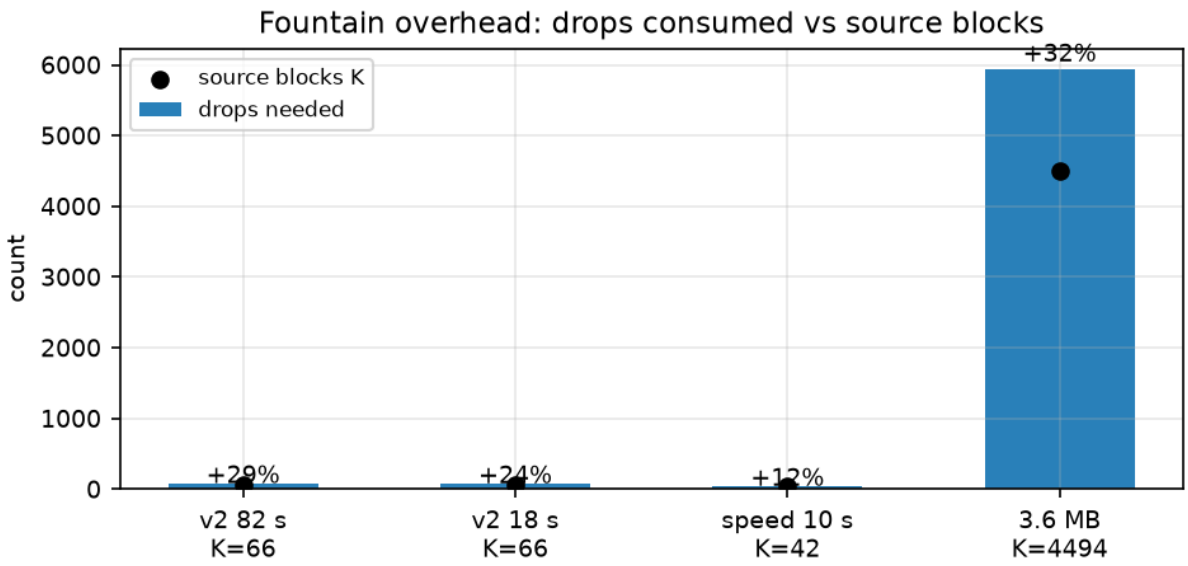


Figure 8 — Fountain overhead stays 12–32% from K = 42 to K = 4,494.

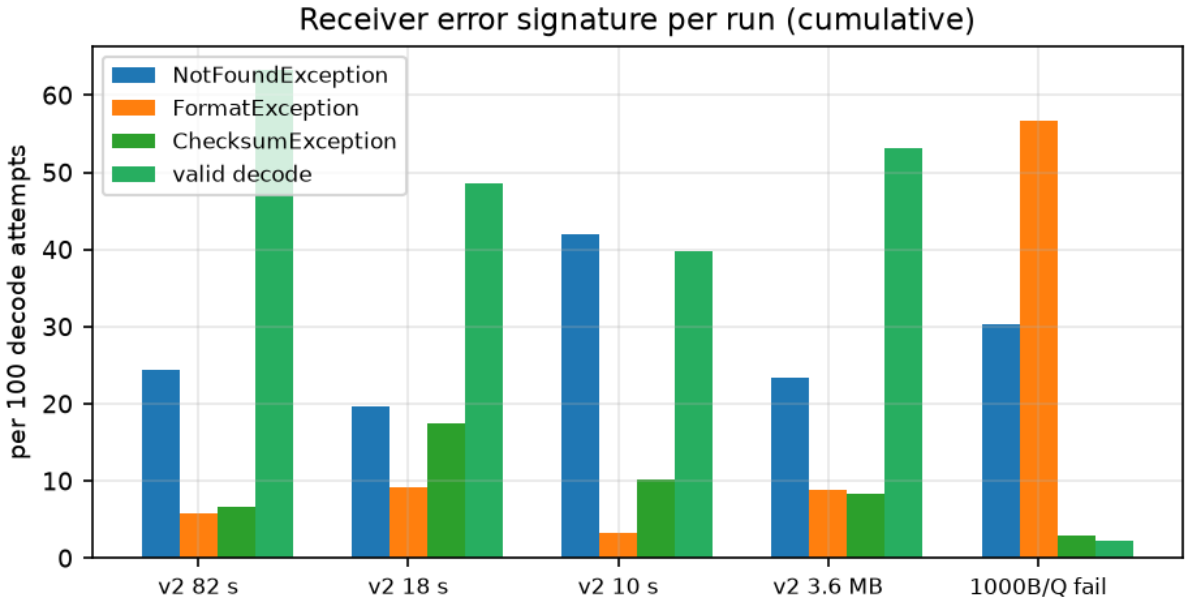


Figure 9 — Per-run decode-error signatures (cumulative, per 100 attempts). The failed 1000B/Q run is dominated by FormatException — the sensor cannot even locate V25 finders — while healthy runs decode ~50% with NotFound (aim/blur) leading.

8.1 Four diagnoses, four fixes

(i) Silent SHA crash. Start-loop did nothing on LAN http://: crypto.subtle is undefined outside secure contexts. Fix: pure-JS SHA-256 fallback + visible error surface. **(ii) Mid-join blindness.** The receiver dropped all data frames until META; joining a 1,000-frame loop mid-way wasted a full cycle. Fix: session keyed by any frame. **(iii) Cursed frames.** Two indices missing for 174 s across ~16 loops at ~40% decode: $P \approx (0.61)^{64} \approx 10^{-14}$ under random-miss hypothesis — proof of systematic undecodability. Root cause: fixed 720-px canvas forced fractional pixels per module, corrupting specific patterns. Fix: integer modules + pixelated upscale; all frames decoded immediately after. **(iv) META hostage.** Data complete in 13 s, DONE 60 s later: META appeared once per 199-frame pass. Fix: META every 20 frames (5% overhead). Next run: 18 s.

Goodput model, validated by the photo run: with ν valid drops/s, block size B and overhead ε,

$$G = \nu \cdot B/(1+\varepsilon) \quad \Rightarrow \quad G \approx 9.7 \cdot 800/1.32 \approx 3.1 \text{ kB/s},$$

matching 3,594,667 B / 1,146 s. Each loop's catch probability per specific frame, explaining the v1 tail, is

$$P = 1 - (1 - 1/N)^A,$$

with A attempts per loop — near 1 for small N, yet v1 still stalled, which is precisely what motivated the 10^{-14} calculation above.

9 Limitations

Optical wall. Block 1000 B at ECC Q (≈V25, 1352 chars) decodes at 2% with FormatException dominating: the test sensor cannot resolve the modules at hand-held distance. The ceiling on *this* hardware is 800 B / ECC M (≈V19); denser codes need better optics, shorter distance, or a stand — not software. **Throughput.** ~3 kB/s net means minutes per megabyte; thermal throttling visibly decays attempt rate after ~3 min of continuous decode. **Security.** Integrity without confidentiality: any line-of-sight camera captures the stream (shoulder-surfable by design); encrypt before encode for sensitive payloads. **Generality.** One phone model, one room, engineering (n = 1) runs — effect sizes are large (22×) but absolute times will shift with hardware. **UX.** Sustained 8–12 fps flashing warrants a photosensitivity warning and a static step-through mode.

10 Future Work

The measured stack leaves clear headroom, in order of expected return: (1) fixed mount + tap-focus lock (attacks the 4.9k NotFounds); (2) RaptorQ / RFC 6330 [7] precode to cut 32% overhead toward ~5%; (3) colour multiplexing after [9] (3× per frame, needs calibration); (4) adaptive bitrate — V15 for bulk, sparse retransmit for the tail; (5) optional encrypted META (X25519 + AES-GCM) for confidential air-gap transfer; (6) multi-receiver broadcast characterisation, the native advantage of a screen over a cable.

11 Conclusion

A QR image stream can carry real files — including multi-megabyte originals — between commodity devices with zero infrastructure, provided three things hold: integer-rendered modules, rateless fountain coding so no frame is special, and metadata frequent enough to never gate completion. The failure modes we met (silent crypto, mid-join blindness, cursed patterns, META hostage, optical wall) were each diagnosed from log statistics and fixed once, and the 22× speedup held on identical hardware. The remaining limit is photons per second, and the logs say exactly how many.

References

[1] ISO/IEC 18004:2024, Information technology — QR code bar code symbology specification. https://www.iso.org/standard/83389.html

[2] DENSO WAVE, “Information capacity and versions of QR Code”. https://www.qrcode.com/en/about/version.html

[3] Thonky, “QR Code Tutorial” (codewords, ECC blocks, mask patterns). https://www.thonky.com/qr-code-tutorial/

[4] M. Luby, “LT Codes”, Proc. 43rd IEEE FOCS, 2002.

[5] J. Byers, M. Luby, M. Mitzenmacher and A. Rege, “A digital fountain approach to reliable distribution of bulk data”, Proc. ACM SIGCOMM, 1998.

[6] A. Shokrollahi, “Raptor codes”, IEEE Trans. Inf. Theory, vol. 52, no. 6, 2006.

[7] M. Luby et al., “RaptorQ Forward Error Correction Scheme”, RFC 6330, 2012. https://www.rfc-editor.org/rfc/rfc6330.txt

[8] “Streaming QR Codes — A Survey”, IEEE (2024). Consolidates 24×/900 kbps streaming results, Xie 3 kbps, Vanderhulst 336 bps–472 kbps, Fath 120.43 kbps. https://ieeexplore.ieee.org/document/10681075

[9] W. Liu et al., “Screen-Camera Communication System Based on Dynamic QR Code”, IOP Conf. Ser. 790, 2020. 150/300 kbps mono, 320/900 kbps colour. https://iopscience.iop.org/article/10.1088/1757-899X/790/1/012012

[10] T. Wang et al., “FareQR: Fast and Reliable Screen-Camera Transfer using QR Code”, IEEE/ACM SEC, 2020. https://www.computer.org/csdl/proceedings-article/sec/2020/594300a364/1rqEzm2yG9q

[11] “RescQR: Enabling Reliable Data Recovery in Screen-Camera Communication”, IEEE Trans. Mobile Computing, vol. 23, 2024. 400+ kbps goodput. https://www.computer.org/csdl/journal/tm/2024/05/10128721/1NdJJqmIv8k

[12] C. Zhang et al., “ChromaCode: A Fully Imperceptible Screen-Camera Communication”, Proc. ACM MobiCom, 2018. https://www.cs.purdue.edu/homes/chunyi/pubs/mobicom18-zhang.pdf

[13] AirCode/ChromaCode slides, USENIX NSDI 2021 (~1 Mbps, 5% BER). https://www.usenix.org/system/files/nsdi21_slides_qian-kun.pdf

[14] ZXing-JS, “@zxing/browser” (BrowserQRCodeReader, decodeFromConstraints). https://github.com/zxing-js/browser/blob/master/README.md

[15] jsQR (pure-JS ZXing QR port), npm. https://www.npmjs.com/package/jsqr/v/0.2.2

[16] “Scan QR codes in browsers with Web Workers and jsQR”, How-To Geek. https://www.howtogeek.com/devops/how-to-scan-qr-codes-in-web-browsers-with-web-workers-and-jsqr

[17] ZXing-JS issue #236: downscaling high-res inputs fixes detection. https://github.com/zxing-js/library/issues/236

[18] UseQR, “How much data fits in a QR code” (ceilings vs scannability). https://useqr.app/docs/spec/how-much-data-fits-in-a-qr-code

[19] “qrcode” npm encoder library. https://www.npmjs.com/package/qrcode

[20] I. Reed and G. Solomon, “Polynomial codes over certain finite fields”, J. SIAM, 1960.

[21] D. MacKay, Information Theory, Inference, and Learning Algorithms, Ch. 50 (fountain codes exposition, free online). https://www.inference.org.uk/mackay/itila/

[22] Google, WebP compression format. https://developers.google.com/speed/webp

[23] NIST FIPS 180-4, Secure Hash Standard (SHA-256). https://doi.org/10.6028/NIST.FIPS.180-4

Reproducibility. Implementation (Vite + TypeScript, MIT libs), full protocol spec, receiver logs (7 sessions) and the scripts that rendered every figure in this paper from those logs are retained with the project. Rerun: npm run dev:https, sender on laptop, receiver on phone via LAN HTTPS.